

UniApp【萌宠案例】pageScrollTo滚动到顶部与刷新功能实现

在萌宠案例的长列表页面（如分类页、推荐页）中，用户浏览大量数据后需快速返回顶部并刷新内容，提升操作效率。本方案基于UniApp原生API `uni.pageScrollTo` 实现滚动到顶部功能，同时联动下拉刷新逻辑，支持“手动触发滚动+自动刷新”“滚动到顶部后触发刷新”两种核心场景，兼顾交互流畅性与功能可靠性。

一、核心技术解析

1. 关键API：uni.pageScrollTo

UniApp提供的页面滚动API，支持将页面滚动到指定位置，核心参数如下，多端兼容性极佳（覆盖小程序、App、H5）：

参数名	类型	必填	说明
scrollTop	Number	是	滚动到页面的目标px），滚动到顶部
duration	Number	否	滚动动画时长（单默认300ms，值越平缓
success	Function	否	滚动成功后的回调于触发后续操作（
fail	Function	否	滚动失败的回调函数常处理

2. 核心逻辑设计

滚动到顶部与刷新功能需实现“触发-滚动-刷新-反馈”的完整链路，核心逻辑分为两类场景：

- 主动触发场景：**用户点击“回到顶部”按钮→调用pageScrollTo滚动→滚动成功后自动触发刷新→显示刷新结果。
- 被动触发场景：**用户滚动页面至底部加载更多后，如需返回顶部刷新→通过滚动监听判断位置，结合下拉刷新逻辑实现联动。



注意：scroll-view组件内的滚动需使用scroll-view的 `scroll-top` 属性控制，与页面级滚动的`pageScrollTo`区分开；本方案同时覆盖页面级滚动与scroll-view滚动两种场景。

二、基础准备：工具类与常量配置

1. 滚动与刷新工具类（utils/scrollUtil.js）

封装滚动到顶部与刷新的通用方法，实现代码复用，统一处理异常。

Code block

```
1  /**
2   * 滚动与刷新工具类
3   * 封装pageScrollTo滚动、scroll-view滚动及联动刷新逻辑
4   */
5  import { asyncUtil } from './asyncUtil'
6
7  export const scrollUtil = {
8    /**
9     * 页面级滚动到顶部
10    * @param {Object} options - 滚动配置
11    * @param {number} options.duration - 滚动时长（默认300ms）
12    * @param {Function} options.success - 滚动成功回调
13    * @param {Function} options.fail - 滚动失败回调
14    * @returns {Promise} - 滚动结果Promise
15    */
16    pageScrollToTop: (options = {}) => {
17      const { duration = 300, success, fail } = options
18      return asyncUtil.promisify(uni.pageScrollTo, {
19        scrollTop: 0,
20        duration,
21        success: (res) => {
22          success && success(res)
23        },
24        fail: (err) => {
25          console.error('页面滚动到顶部失败: ', err)
26          fail && fail(err)
27          throw err
28        }
29      })
30    },
31
32    /**
```

```

33      * scroll-view滚动到顶部
34      * @param {Object} scrollRef - scroll-view的ref对象
35      * @param {number} duration - 滚动时长 (默认300ms)
36      */
37      scrollViewToTop: (scrollRef, duration = 300) => {
38          if (!scrollRef || !scrollRef.value) {
39              console.error('scroll-view ref不存在')
40              return Promise.reject(new Error('scroll-view引用异常'))
41          }
42          // 调用scroll-view的滚动方法
43          return new Promise((resolve) => {
44              scrollRef.value.scrollTo({
45                  scrollTop: 0,
46                  duration,
47                  success: () => resolve(true),
48                  fail: (err) => {
49                      console.error('scroll-view滚动失败: ', err)
50                      resolve(false)
51                  }
52              })
53          })
54      },
55
56      /**
57      * 滚动后触发刷新
58      * @param {Function} refreshFn - 刷新函数
59      * @param {Object} scrollOptions - 滚动配置
60      * @returns {Promise} - 滚动+刷新的组合结果
61      */
62      scrollThenRefresh: async (refreshFn, scrollOptions = {}) => {
63          if (typeof refreshFn !== 'function') {
64              throw new Error('刷新函数必须为Function类型')
65          }
66          // 先滚动到顶部, 成功后执行刷新
67          await scrollUtil.pageScrollToTop(scrollOptions)
68          // 延迟100ms执行刷新, 确保滚动动画完成
69          return new Promise((resolve) => {
70              setTimeout(async () => {
71                  try {
72                      const refreshRes = await refreshFn()
73                      resolve({ scrollSuccess: true, refreshSuccess: true, data:
refreshRes })
74                  } catch (err) {
75                      resolve({ scrollSuccess: true, refreshSuccess: false, error: err })
76                  }
77              }, 100)
78          })

```

```
79   }  
80 }
```

2. 常量配置 (utils/constant.js)

统一管理滚动与刷新相关常量，便于维护。

Code block

```
1  export const Constant = {  
2    // 滚动配置  
3    SCROLL: {  
4      DURATION: 300, // 默认滚动时长 (ms)  
5      FAST_DURATION: 150, // 快速滚动时长 (ms)  
6      SHOW_BACK_TOP_THRESHOLD: 500 // 显示回到顶部按钮的阈值 (px)  
7    },  
8    // 刷新配置  
9    REFRESH: {  
10     DELAY: 100 // 滚动后刷新的延迟时间 (ms)  
11   },  
12   // 接口状态  
13   API_CODE: {  
14     SUCCESS: 0  
15   }  
16 }
```

三、核心场景实现：分类页 (scroll-view场景)

萌宠分类页使用scroll-view作为列表容器，需实现“滚动监听显示回到顶部按钮→点击按钮滚动并刷新→刷新反馈”的完整流程。

1. 页面结构 (pages/category/category.vue)

包含scroll-view列表、回到顶部按钮、刷新状态提示，通过ref关联scroll-view组件。

Code block

```
1  <template>  
2    <view class="category-page">  
3  
4      <view class="category-tabs">  
5        <view  
6          class="tab-item"  
7          v-for="(tab, index) in categoryTabs"  
8          :key="tab.id">
```

```
9      :class="{ active: currentTab === index }"
10      @tap="switchTab(index)"
11    >
12      {{ tab.name }}
13    </view>
14  </view>
15
16
17  <scroll-view
18    ref="petScrollRef"
19    class="pet-scroll-view"
20    scroll-y
21    refresher-enabled
22    @refresherrefresh="onPullRefresh"
23    @scrolltolower="onReachBottom"
24    @scroll="onScroll"
25  >
26    <view class="pet-list">
27      <navigator
28        class="pet-item pet-card"
29        v-for="(pet, index) in petList"
30        :key="pet.id + index"
31        :url="'/pages/detail/detail?id=' + pet.id"
32      >
33        <image
34          :src="pet.coverImg"
35          mode="widthFix"
36          class="pet-img"
37          lazy-load
38          placeholder-image="/static/placeholder.png"
39        ></image>
40        <view class="pet-info">
41          <view class="pet-name">{{ pet.name }}</view>
42          <view class="pet-breed">品种: {{ pet.breed }}</view>
43          <view class="pet-meta">
44            <view class="like-count">{{ pet.likeCount }}</view>
45            <view class="view-count">浏览 {{ pet.viewCount }}</view>
46          </view>
47        </navigator>
48      </view>
49
50
51      <view class="loading-tip" v-if="isLoadingMore">加载中...</view>
52      <view class="loading-tip" v-if="noMoreData">已加载全部数据</view>
53    </view>
54  </scroll-view>
55
```

```

56
57     <view
58       class="back-top-btn"
59       v-if="showBackTop"
60       @tap="handleBackTop"
61       :class="{ active: isScrolling }"
62     >
63       <image src="/static/icon/back-top.png" mode="widthFix" class="btn-icon">
64     </image>
65     <view class="btn-text">顶部</view>
66   </view>
67
68   <view class="refresh-toast" v-if="refreshToast">{{ refreshToast }}</view>
69 </view>
70 </template>

```

2. 逻辑实现 (pages/category/category.vue)

核心逻辑包含滚动监听、回到顶部 (scroll-view)、滚动后刷新、分页加载等，严格管理各状态避免冲突。

Code block

```

1  <script setup>
2  import { ref, onLoad, onUnload } from 'vue'
3  import request from '../utils/request'
4  import { scrollUtil } from '../utils/scrollUtil'
5  import { Constant } from '../utils/constant'
6
7  // 分类数据
8  const categoryTabs = ref([
9    { id: 1, name: '全部', type: 'all' },
10   { id: 2, name: '猫咪', type: 'cat' },
11   { id: 3, name: '狗狗', type: 'dog' },
12   { id: 4, name: '其他', type: 'other' }
13 ])
14 const currentTab = ref(0)
15
16 // 列表与分页状态
17 const petList = ref([])
18 const page = ref(1)
19 const pageSize = ref(10)
20 const isLoadingMore = ref(false)
21 const noMoreData = ref(false)
22 const refreshToast = ref('')

```

```
23
24 // 滚动相关状态
25 const petScrollRef = ref(null) // scroll-view的ref
26 const showBackTop = ref(false) // 是否显示回到顶部按钮
27 const isScrolling = ref(false) // 是否正在滚动
28 const scrollTop = ref(0) // 当前滚动位置
29
30 // 页面加载：初始化数据
31 onLoad(() => {
32   loadPetData()
33 })
34
35 // 页面卸载：清除定时器
36 onUnload(() => {
37   if (refreshToastTimer.value) {
38     clearTimeout(refreshToastTimer.value)
39   }
40 })
41
42 // 加载萌宠数据
43 const loadPetData = () => {
44   const currentType = categoryTabs.value[currentTab.value].type
45   const params = { type: currentType, page: page.value, pageSize:
46     pageSize.value }
47   return request.get('/category/pet', params)
48     .then((res) => {
49       if (!Array.isArray(res.list)) {
50         throw new Error('列表数据格式异常')
51       }
52       const validList = res.list.filter(item => item.id && item.coverImg)
53       if (page.value === 1) {
54         petList.value = validList
55       } else {
56         petList.value = [...petList.value, ...validList]
57       }
58       noMoreData.value = validList.length < pageSize.value
59       return validList
60     })
61     .catch((errorInfo) => {
62       if (page.value > 1) page.value -= 1
63       showRefreshToast(errorInfo.msg, false)
64       throw errorInfo
65     })
66   }
67
68 // 滚动监听：控制回到顶部按钮显示/隐藏
```

```

69  const onScroll = (e) => {
70    scrollTop.value = e.detail.scrollTop
71    // 滚动超过阈值显示按钮
72    showBackTop.value = scrollTop.value >=
    Constant.SCROLL.SHOW_BACK_TOP_THRESHOLD
73  }
74
75  // 处理回到顶部：滚动+刷新
76  const handleBackTop = () => {
77    if (isScrolling.value) return
78    isScrolling.value = true
79    // 先滚动到顶部，再触发刷新
80    scrollUtil.scrollToTop(petScrollRef, Constant.SCROLL.DURATION)
81    .then((scrollSuccess) => {
82      if (scrollSuccess) {
83        // 滚动成功后执行刷新
84        return loadPetData()
85      } else {
86        throw new Error('滚动失败')

```

3. 样式实现 (pages/category/category.vue)

适配回到顶部按钮的显示/隐藏动画、滚动状态反馈，确保交互体验流畅。

Code block

```

1  .category-page {
2    background-color: #f5f5f5;
3    min-height: 100vh;
4    position: relative;
5  }
6
7  /* 分类标签栏 */
8  .category-tabs {
9    display: flex;
10   justify-content: space-around;
11   align-items: center;
12   height: 88rpx;
13   background-color: #fff;
14   border-bottom: 1rpx solid #eee;
15   position: sticky;
16   top: 0;
17   z-index: 10;
18 }
19 .tab-item {
20   font-size: 28rpx;
21   color: #666;
22   padding: 0 10rpx;

```



```
23     height: 100%;
24     display: flex;
25     align-items: center;
26 }
27 .tab-item.active {
28     color: #ff6700;
29     font-weight: 500;
30     position: relative;
31 }
32 .tab-item.active::after {
33     content: '';
34     position: absolute;
35     bottom: 0;
36     left: 0;
37     width: 100%;
38     height: 6rpx;
39     background-color: #ff6700;
40     border-radius: 3rpx;
41 }
42
43 /* scroll-view列表 */
44 .pet-scroll-view {
45     width: 100%;
46     height: calc(100vh - 88rpx);
47     padding: 24rpx;
48     box-sizing: border-box;
49 }
50 .pet-list {
51     display: flex;
52     flex-wrap: wrap;
53     justify-content: space-between;
54     gap: 24rpx;
55 }
56 .pet-item {
57     width: calc(50% - 12rpx);
58     background-color: #fff;
59     border-radius: 16rpx;
60     overflow: hidden;
61     box-shadow: 0 2rpx 8rpx rgba(0, 0, 0, 0.05);
62 }
63 .pet-img {
64     width: 100%;
65     height: 280rpx;
66 }
67 .pet-info {
68     padding: 20rpx;
69 }
```

```
70 .pet-name {
71   font-size: 28rpx;
72   margin-bottom: 8rpx;
73 }
74 .pet-breed {
75   font-size: 24rpx;
76   color: #666;
77   margin-bottom: 16rpx;
78 }
79 .pet-meta {
80   display: flex;
81   justify-content: space-between;
82   font-size: 22rpx;
83   color: #999;
84 }
85
86 /* 回到顶部按钮 */
87 .back-top-btn {
88   position: fixed;
```

四、扩展场景：推荐页（页面级滚动场景）

萌宠推荐页采用页面级滚动（非scroll-view），需通过pageScrollTo实现滚动到顶部，同时联动页面的下拉刷新逻辑。

1. 页面结构与逻辑（pages/recommend/recommend.vue）

Code block

```
1 <template>
2   <view class="recommend-page">
3
4     <view class="search-bar">
5       <view class="search-input">
6         <image src="/static/icon/search.png" mode="widthFix" class="search-
icon"></image>
7         <input placeholder="搜索萌宠名称..." />
8       </view>
9     </view>
10
11
12     <view class="recommend-list">
13       <view class="section-header">
14         <text class="header-title">为你推荐</text>
15       </view>
16       <navigator
17         class="pet-item">
```

```
18         v-for="(pet, index) in petList"
19         :key="pet.id"
20         :url="'/pages/detail/detail?id=' + pet.id"
21     >
22         <image :src="pet.coverImg" mode="widthFix" class="pet-img"></image>
23         <view class="pet-info">{{ pet.name }}</view>
24     </navigator>
25 </view>
26
27
28 <view
29     class="back-top-btn"
30     v-if="showBackTop"
31     @tap="handleBackTop"
32 >
33     顶部
34 </view>
35 </view>
36 </template>
37
38 <script setup>
39 import { ref, onPageScroll } from 'vue'
40 import request from '../utils/request'
41 import { scrollUtil } from '../utils/scrollUtil'
42 import { Constant } from '../utils/constant'
43
44 // 列表数据与状态
45 const petList = ref([])
46 const isRefreshing = ref(false)
47 const showBackTop = ref(false)
48
49 // 页面滚动监听（页面级滚动）
50 onPageScroll((e) => {
51     // 控制回到顶部按钮显示
52     showBackTop.value = e.scrollTop >= Constant.SCROLL.SHOW_BACK_TOP_THRESHOLD
53 })
54
55 // 加载推荐数据
56 const loadRecommendData = () => {
57     isRefreshing.value = true
58     return request.get('/pet/recommend')
59         .then((res) => {
60             if (Array.isArray(res.list)) {
61                 petList.value = res.list.filter(item => item.id)
62             }
63             return res.list
64         })
65 }
```

```

65     .catch((errorInfo) => {
66         uni.showToast({ title: errorInfo.msg, icon: 'error' })
67         throw errorInfo
68     })
69     .finally(() => {
70         isRefreshing.value = false
71         // 关闭页面级下拉刷新动画
72         uni.stopPullDownRefresh()
73     })
74 }
75
76 // 页面加载时初始化
77 loadRecommendData()
78
79 // 处理回到顶部：滚动+刷新
80 const handleBackTop = () => {
81     // 使用工具类的scrollThenRefresh方法，滚动后自动刷新
82     scrollUtil.scrollThenRefresh(loadRecommendData, {
83         duration: Constant.SCROLL.DURATION
84     })
85     .then((result) => {
86         if (result.refreshSuccess) {
87             uni.showToast({ title: '刷新成功', icon: 'success' })

```

五、多端适配与异常处理

1. 多端适配要点

- **小程序端：**

页面级滚动的pageScrollTo在小程序中需确保页面无overflow:hidden样式，否则可能失效。

- scroll-view的scrollTo方法在微信小程序基础库 $\geq 2.12.0$ 中支持，低版本需兼容处理。

App端：

Android端页面级滚动可能受软键盘弹出影响，需在滚动前判断软键盘状态并关闭。

iOS端滚动动画时长建议不超过500ms，避免滚动过于缓慢影响体验。

H5端：

H5端页面级滚动的scrollTop值可能受浏览器导航栏影响，需通过window.innerHeight校准。

低版本浏览器不支持pageScrollTo的duration参数，需降级处理为无动画滚动。

2. 异常处理方案

- **滚动失败处理：**在scrollUtil中捕获滚动异常，通过toast提示用户“滚动失败，请重试”，同时允许用户手动触发刷新。

- **刷新失败处理**：滚动成功但刷新失败时，显示错误提示，保留原有列表数据，避免用户无数据可看。
- **ref丢失处理**：scroll-view场景中，若ref未正确关联，提示开发者检查组件引用，同时降级为页面级滚动。
- **重复触发处理**：通过isScrolling状态锁，防止用户快速点击回到顶部按钮导致的重复滚动与刷新。

六、性能优化技巧

1. **滚动节流**：页面级滚动的onPageScroll事件触发频繁，可添加节流（如200ms）减少性能消耗：

```
// 节流函数
const throttle = (fn, delay = 200) => {
  let timer = null
  return (...args) => {
    if (!timer) {
      timer = setTimeout(() => {
        fn(...args)
        timer = null
      }, delay)
    }
  }
}

// 节流后的滚动监听
onPageScroll(throttle((e) => {
  showBackTop.value = e.scrollTop >=
Constant.SCROLL.SHOW_BACK_TOP_THRESHOLD
})))
```

2. **条件渲染优化**：回到顶部按钮的显示/隐藏通过v-if控制，而非opacity，减少页面渲染节点。
3. **滚动动画适配**：根据设备性能动态调整滚动时长，低端设备使用快速滚动（150ms），高端设备使用默认时长。
4. **预加载数据**：刷新前可预加载部分数据，减少用户等待时间，提升刷新体验。

七、常见问题与解决方案

问题现象	原因分析	解决方案
pageScrollTo无效果	1. 页面有overflow:hidden样式；2. scrollTop值设置错	1. 移除页面的overflow:hidden；2. 确认

	误；3. 小程序基础库版本过低	scrollTop设为0；3. 升级小程序基础库至2.0以上
scroll-view滚动后刷新不触发	1. scrollRef未正确关联组件；2. 滚动成功后未调用刷新函数	1. 检查ref属性与代码中的引用是否一致；2. 在scrollViewToTop的then中调用刷新函数
回到顶部按钮频繁显示/隐藏	滚动监听未节流，滚动到阈值附近时频繁触发状态变化	为滚动监听事件添加节流（200ms），减少状态更新频率
滚动动画与刷新衔接生硬	滚动完成后立即执行刷新，无时间间隔	在滚动成功后延迟100ms执行刷新，确保滚动动画完成

八、总结

本方案基于UniApp原生能力，为萌宠案例实现了覆盖“scroll-view”与“页面级滚动”的滚动到顶部与刷新功能，核心价值如下：

- 1. **场景全覆盖**：同时支持列表页的两种滚动模式，适配不同页面的开发需求，代码可复用性高。
- 2. **交互体验优**：滚动动画平缓，回到顶部按钮按需显示，刷新有明确反馈，符合用户操作习惯。
- 3. **多端兼容好**：针对小程序、App、H5端的特性进行适配，避免跨端问题。
- 4. **状态可控**：通过状态锁、节流等手段避免重复操作，确保滚动与刷新逻辑稳定可靠。

该方案可根据业务需求进一步扩展，如添加“滚动到指定位置”“滚动进度条”等功能，或结合用户行为数据优化滚动阈值与动画时长，提升整体交互体验。

（注：文档部分内容可能由 AI 生成）